FPGA Workshop
2. Diving Deeper

Sponsored by Jane Street

Contents

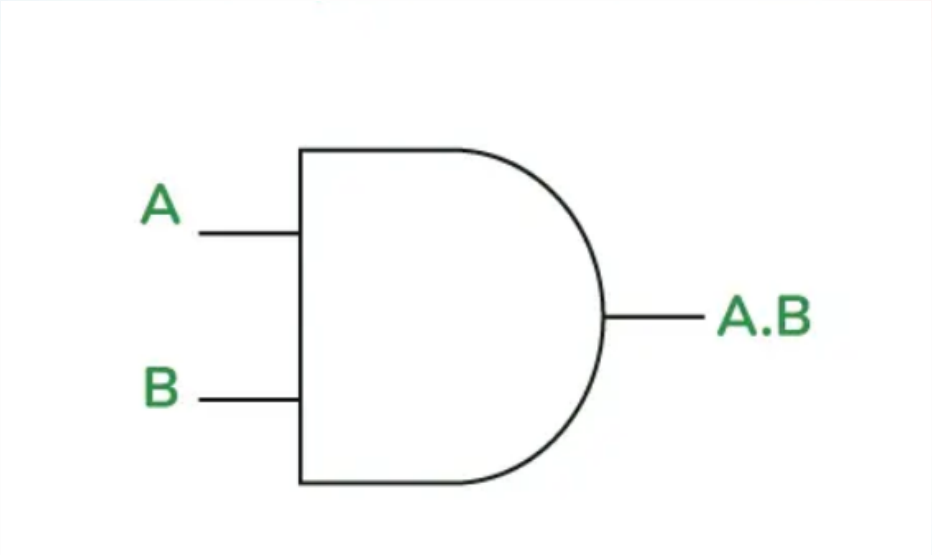
1. RECAP + small yap
2. Verilog Logic Gates
3. Seven Segment Displays
4. The 50MHz Problem
5. Stopwatch
6. Homework

1. Recap

We covered a lot last week

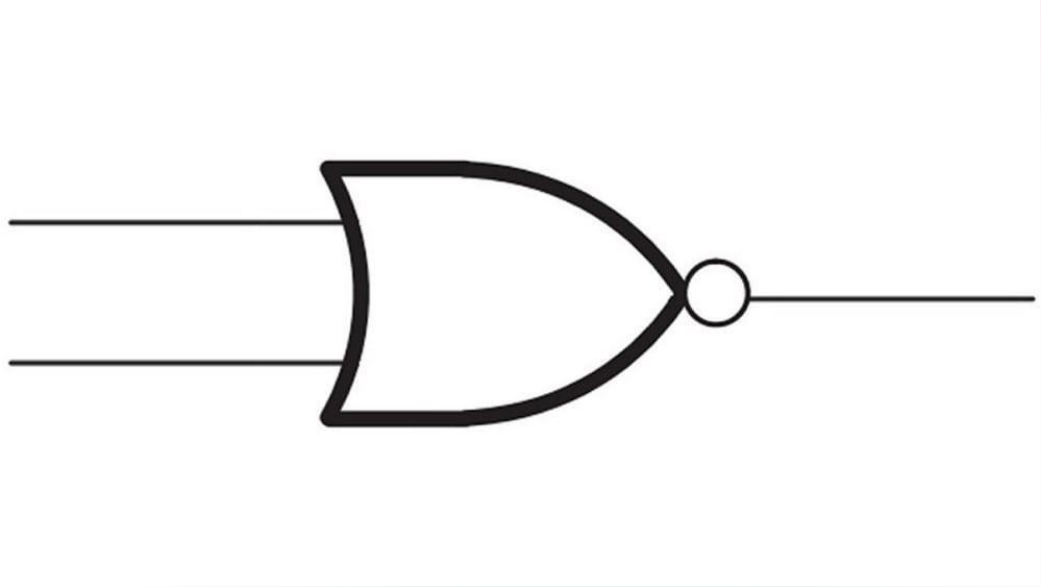
DONUT TIME!

AND

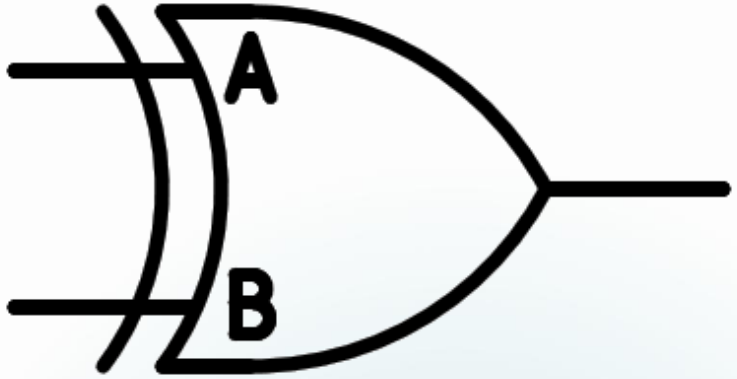


OR

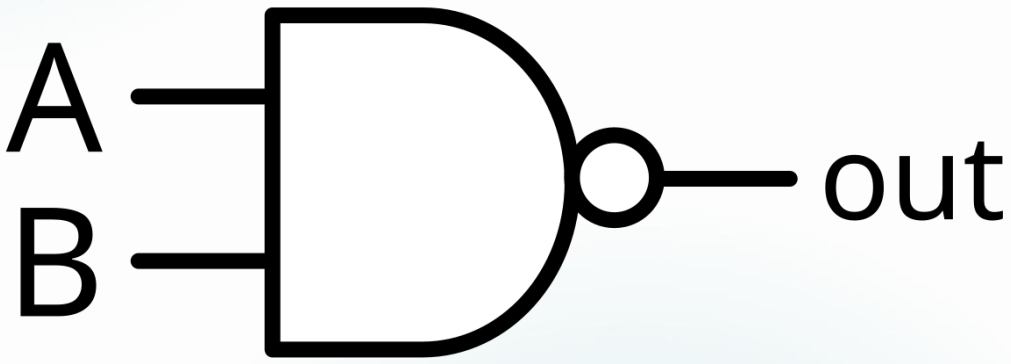
NOR



XOR



NAND



BINARY TO DENARY

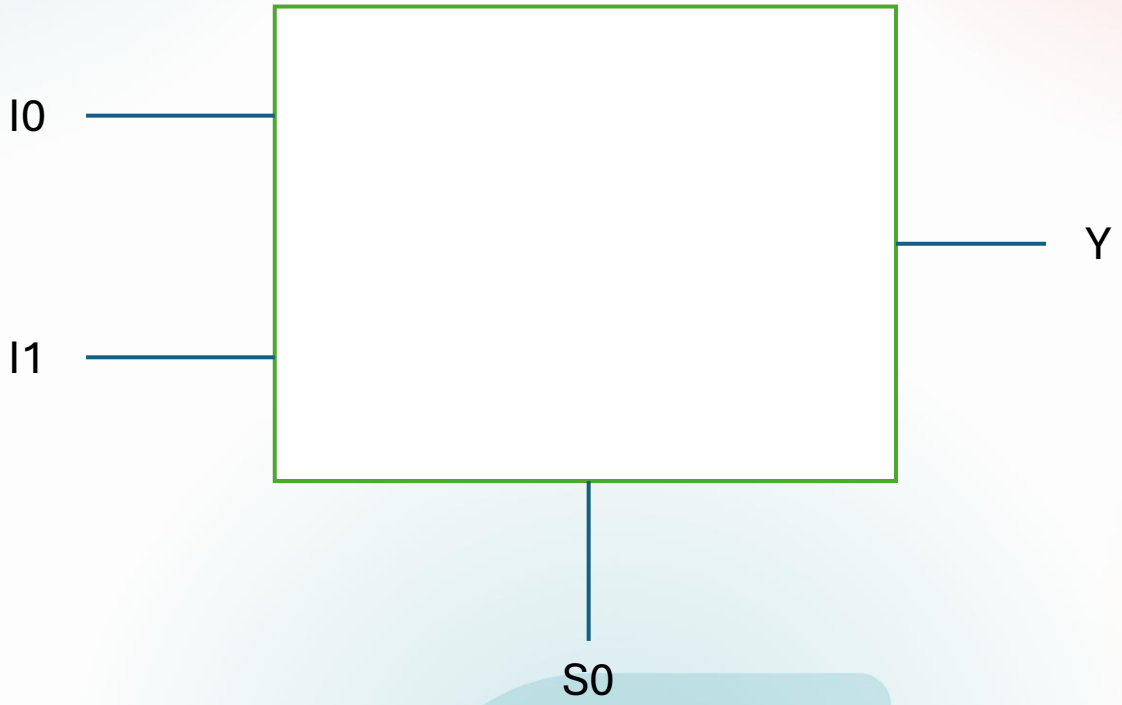
01110101

DENARY TO BINARY

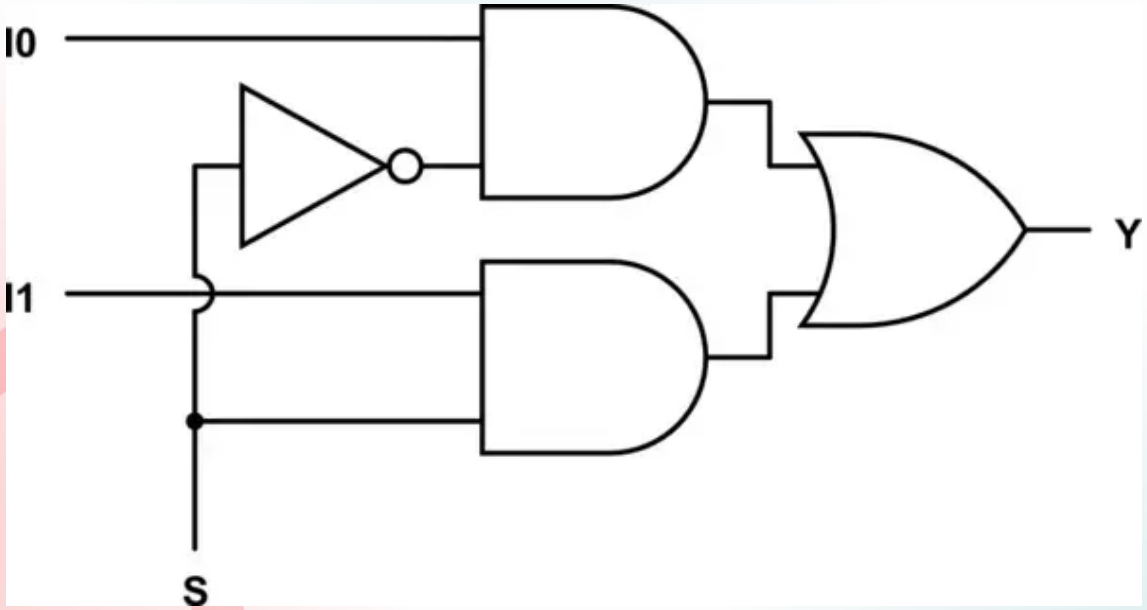
67

THE FINAL BOSS

2:1 Multiplexer

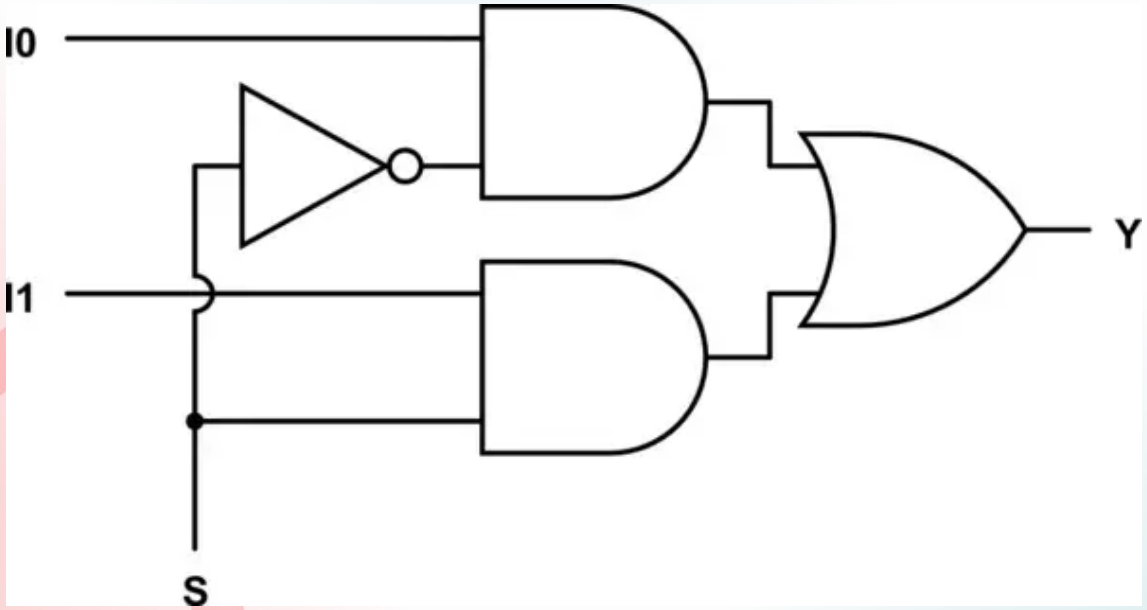


2:1 Multiplexer



I0	I1	S0	Yk	Yu
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

2:1 Multiplexer Answers



I0	I1	S0	Yk	Yu
0	0	0	0	I0
0	0	1	0	I1
0	1	0	0	I0
0	1	1	1	I0
1	0	0	1	I1
1	0	1	0	I0
1	1	0	1	I1
1	1	1	1	I0

Importance?

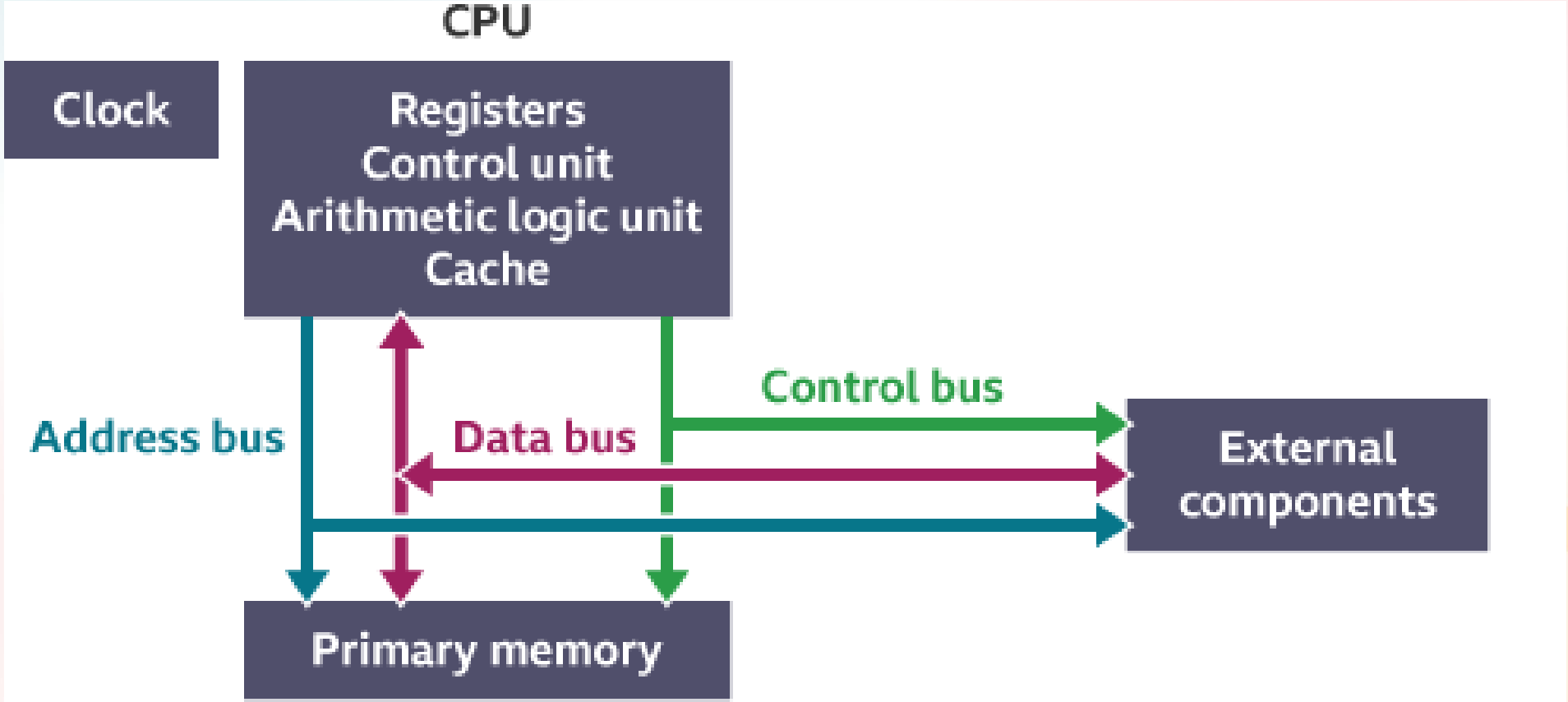
They act like a **switch** that decides which signal moves forward in the circuit.
Essentially hardware for an if statement

```
Assign y = s ? b:a;
```

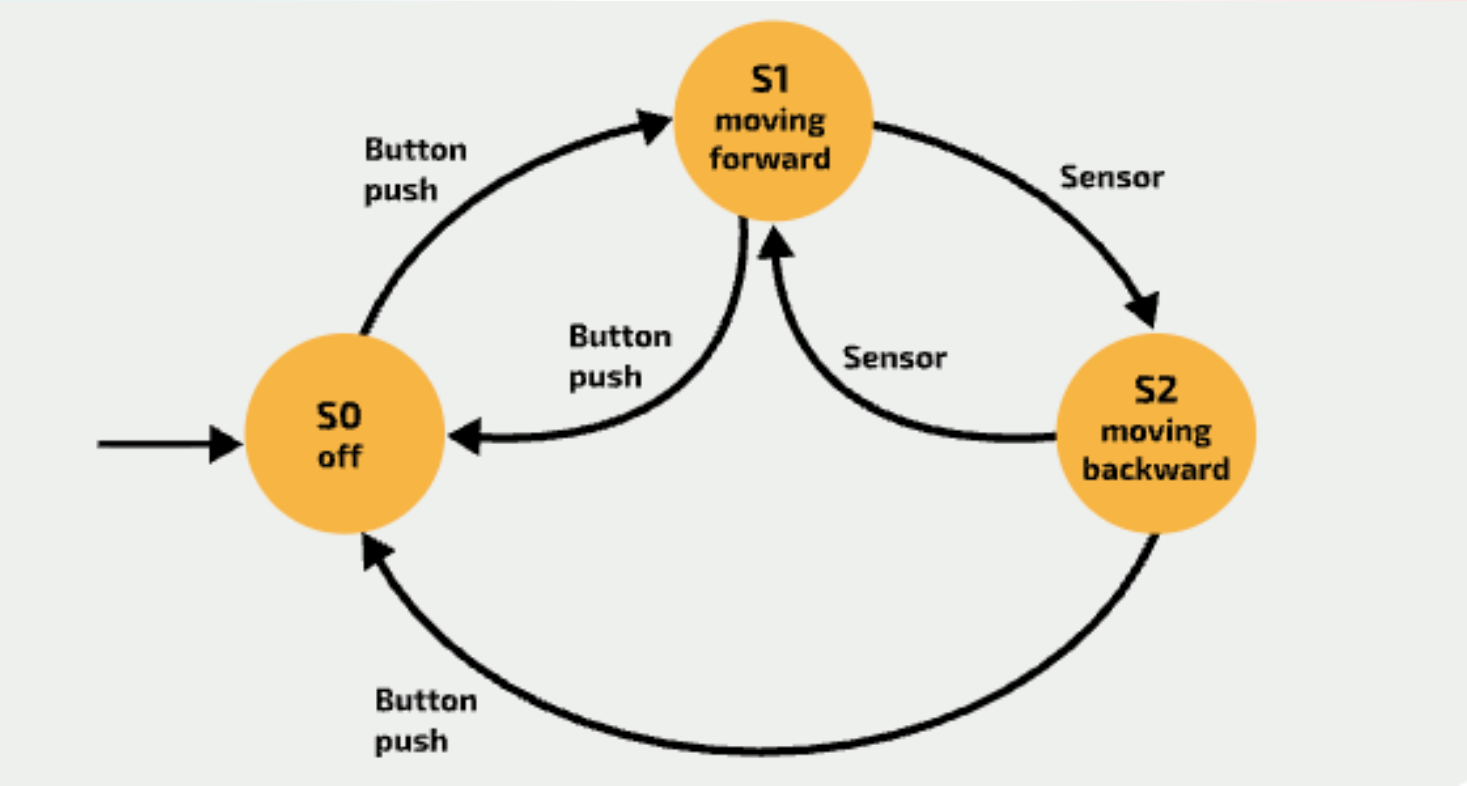
```
If (s)
 y = b;
else:
y=a;
```

Even complex logic often compiles down to **many small multiplexers**.

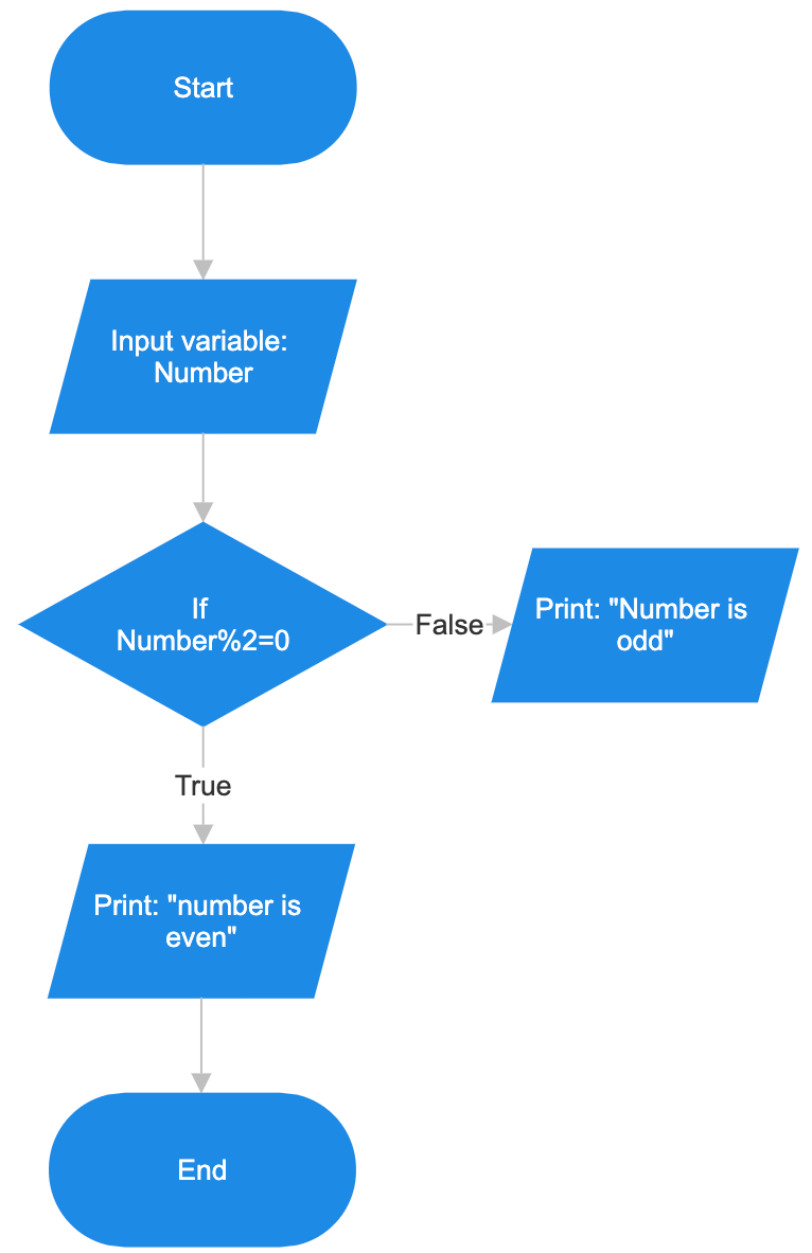
Inside a computer



Finite State Machines



Flowcharts


Start
Input variable: Number
If Number%2=0
False
Print: "Number is odd"
True
Print: "number is even"
End

Enough Yap

2. Verilog Logic Gates

1. Verilog SR

- An SR latch in Verilog

```
module sr_latch (
    input S, // Set (Make the light 1)
    input R, // Reset (Make the light 0)
    output Q, // The stored bit
    output Qn // The opposite of the bit
);
    // These are just raw wires "feeding
back" into the gates
    assign Q  = ~(R | Qn);
    assign Qn = ~(S | Q);
endmodule
```

2. Blocking vs Non-Blocking

2. Everything, Everywhere, all at once.

- In our hardware **everything happens at the same time.**
- **Verilog assignment is <u>non-blocking</u>**

2. Everything, Everywhere, all at once.

In high level – one line at a time.

```
while True:
    # Step 1: Check Motor
    if start_button and safety_key:
        motor = ON
    else:
        motor = OFF

    # Step 2: Check Temp (The Chef only
gets here AFTER checking the motor)
    if temperature > 100:
        warning_led = ON
```

In Verilog, all at same time.

```
assign motor = start_button & safety_key;
assign warning_led = (temperature > 100);
```

2. Everything, Everywhere, all at once.

Why is this the case?

```
assign motor = start_button & safety_key;
assign warning_led = (temperature > 100);
```

Because in Verilog, you're not **writing instructions.** You're **wiring components.**

2. Everything, Everywhere, all at once.

Why is this the case?

```
assign motor = start_button & safety_key;
assign warning_led = (temperature > 100);
```

Because in Verilog, you're not **writing instructions.** You're **wiring components.**

2. Everything, Everywhere, all at once.

Let's pick this apart:

```
assign motor = start_button & safety_key;
assign warning_led = (temperature > 100);
```

& - AND

| - OR

^ XOR

~ NOT

In Verilog:

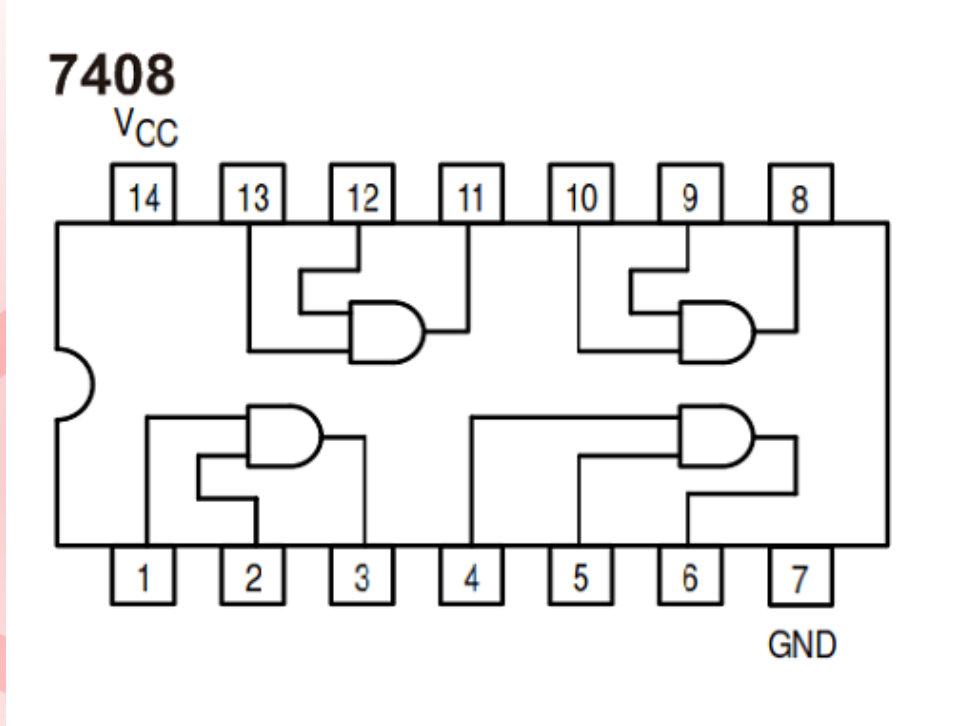
```
assign Y = A & B;
```

Behaves as a **solder joint**.

2. Long ago, the four nations lived together in harmony.

Before FPGA's Existed, you'd have to buy IC's with specific purposes.

e.g. Here's a 7408:



When you write in Verilog:

```
assign Y = A & B;
```

'&' tells the FPGA etcher to

1. **Configure** a bit of internal silicon to behave like an AND gate.
2. Connect A,B wires as input
3. Connect Y wire as output.

2. Long ago, the four nations lived together in harmony.

- Everything in FPGA's is built from Transistors
- They're all wired up into the LUT

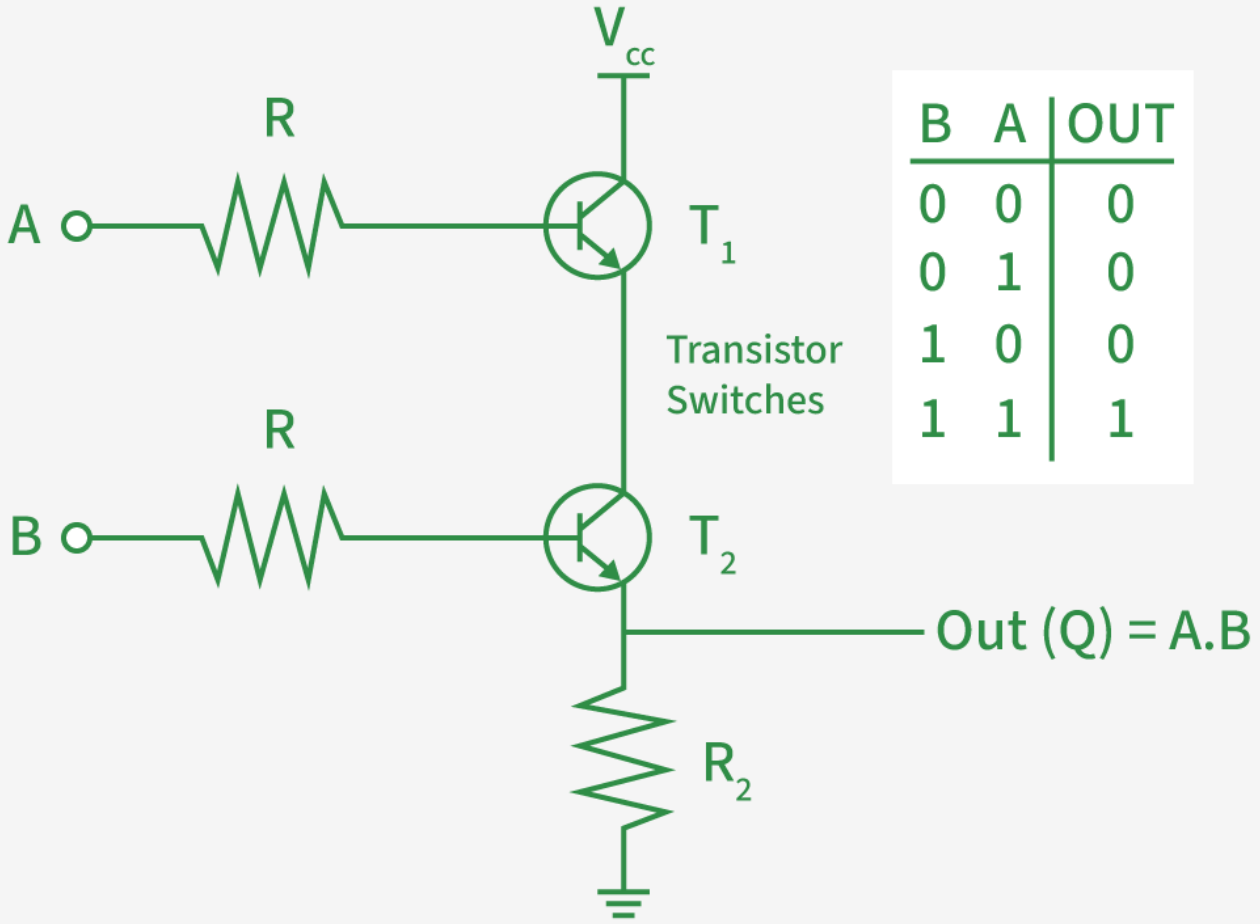
- When you flash an FPGA, you flash the LUT.
- When you

```
assign Y = A & B;
```

  The FPGA "programs" the LUT with our truth table for AND.
  The FPGA "links" wires A,B and Y to the truth table.

- This is the same for AND, OR, NOT, XOR

2. The Logical Song



B	A	OUT
0	0	0
0	1	0
1	0	0
1	1	1

2. The Look (up table)

Because we're using **LUT transistors** and not high level logic

- We have real physical limits
- **Propagation delay:** It takes a fraction of a nanosecond for electricity to pass through the LUT
- **Heat**: Heat is created when we flip from 0 to 1 and 1 to 0

2. The Look (up table)

This is much faster than C++

In C++

- You compile to get **machine code**
- **Machine code** is a list of fixed instructions for a general purpose CPU
- The CPU reads an instruction, goes to memory, brings data back to registers, does the maths and then writes back.

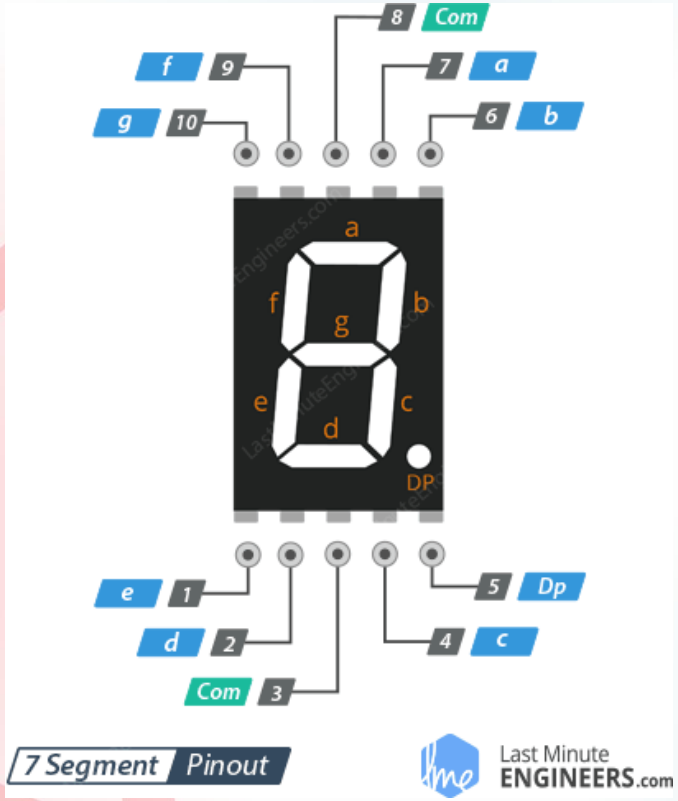
In Verilog

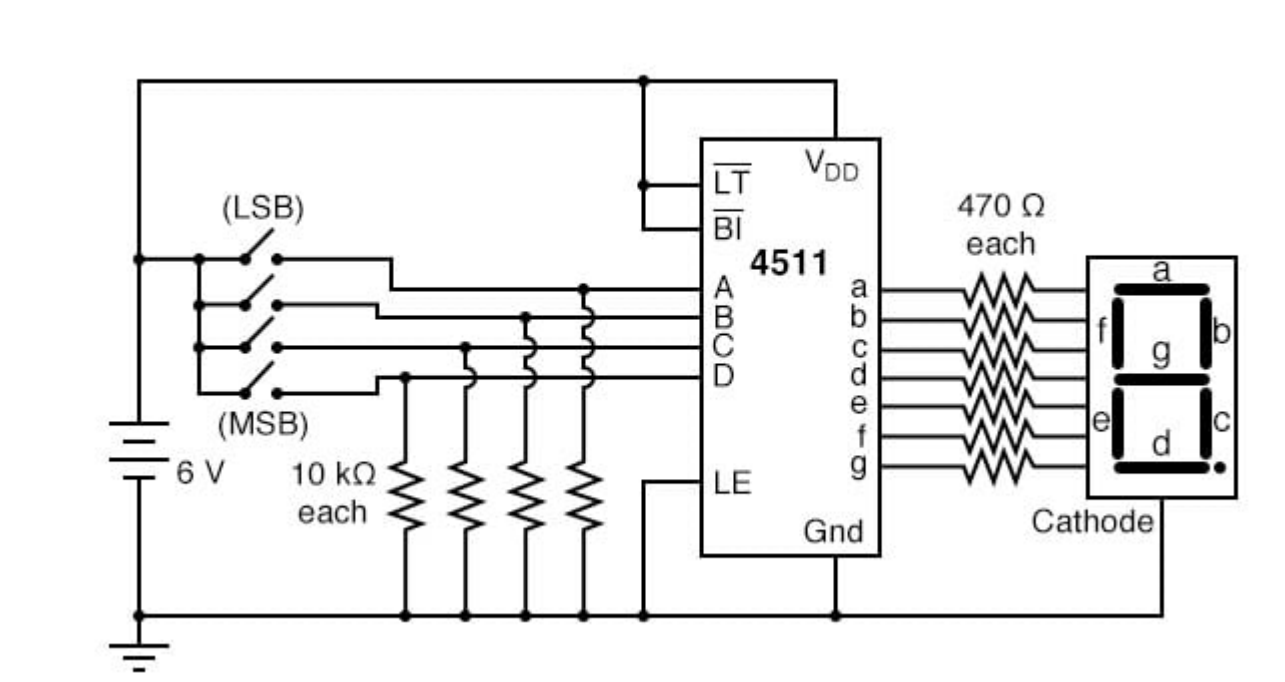
- You draw an AND gate.
- It physically exists.
- You just put electricity through the goddamn wires.

3. Seven Segment Displays

3. The Seven Nation Army

A 7 Segment Display is just 7 LED's
that share a common ground/power.

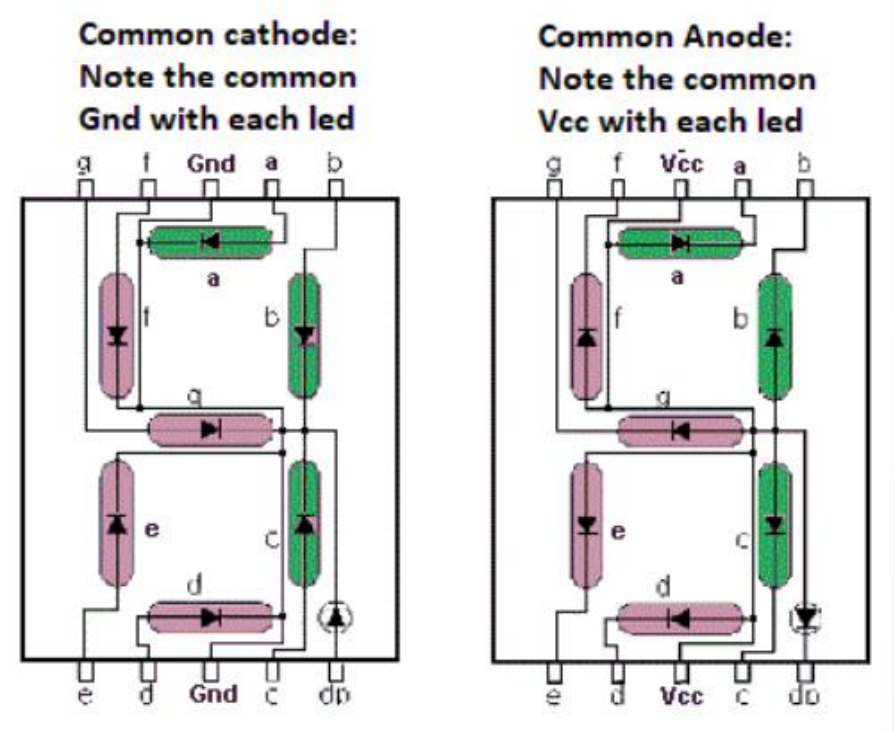


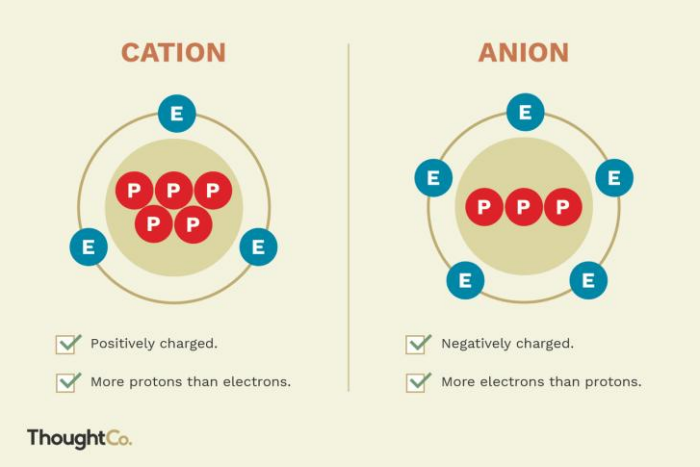


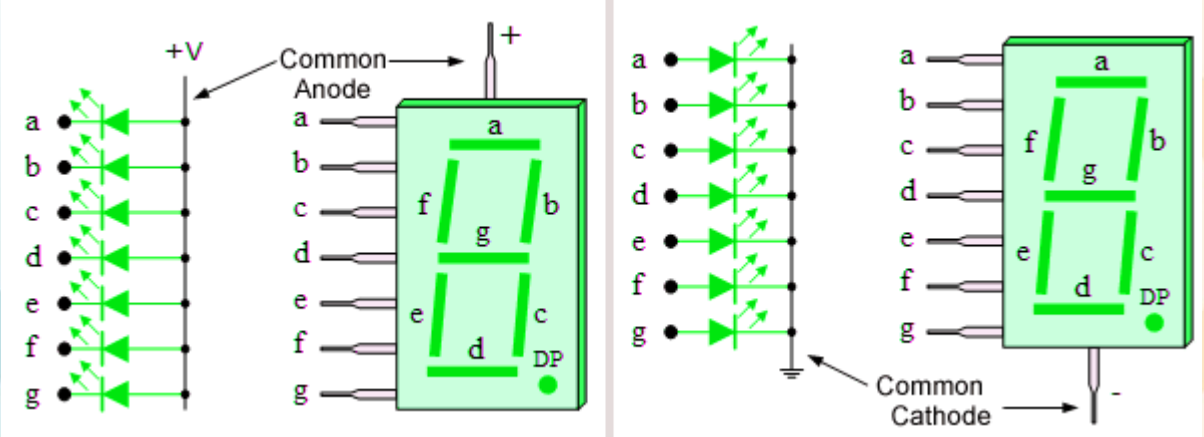
3. The Seven Nation Army

They can be **common anode – 0** turns segment OFF

or **common cathode – 1** turns segment ON





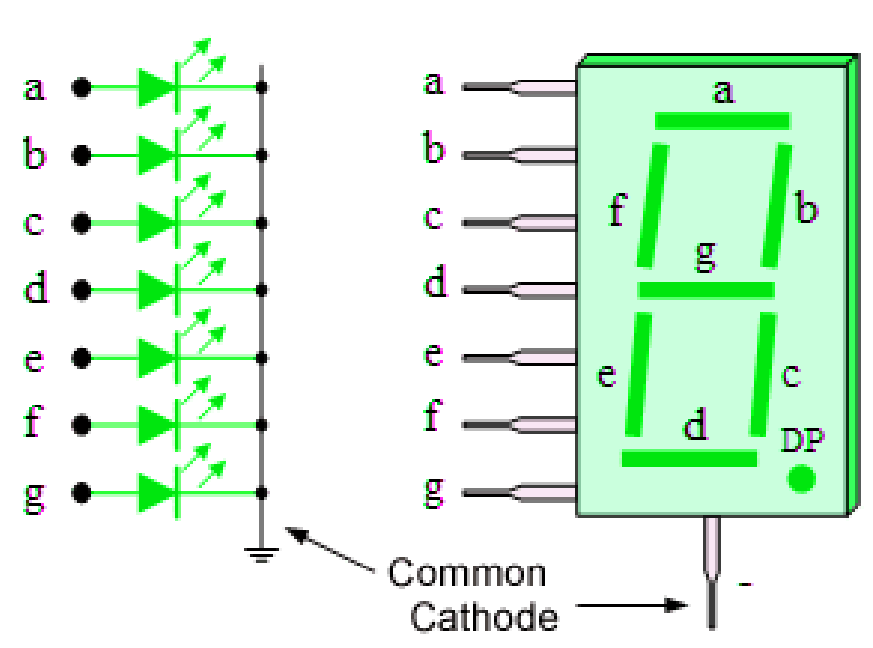


3. The Seven Nation Army

Let's focus on **Common Cathode**.

- Every segment is labeled **a** through **g**.
- We usually represent them as a 7-bit bus: [6:0] segments.

```
seg[0] = a
seg[1] = b
seg[2] = c
seg[3] = d
seg[4] = e
seg[5] = f
seg[6] = g
```



3. The Seven Nation Army

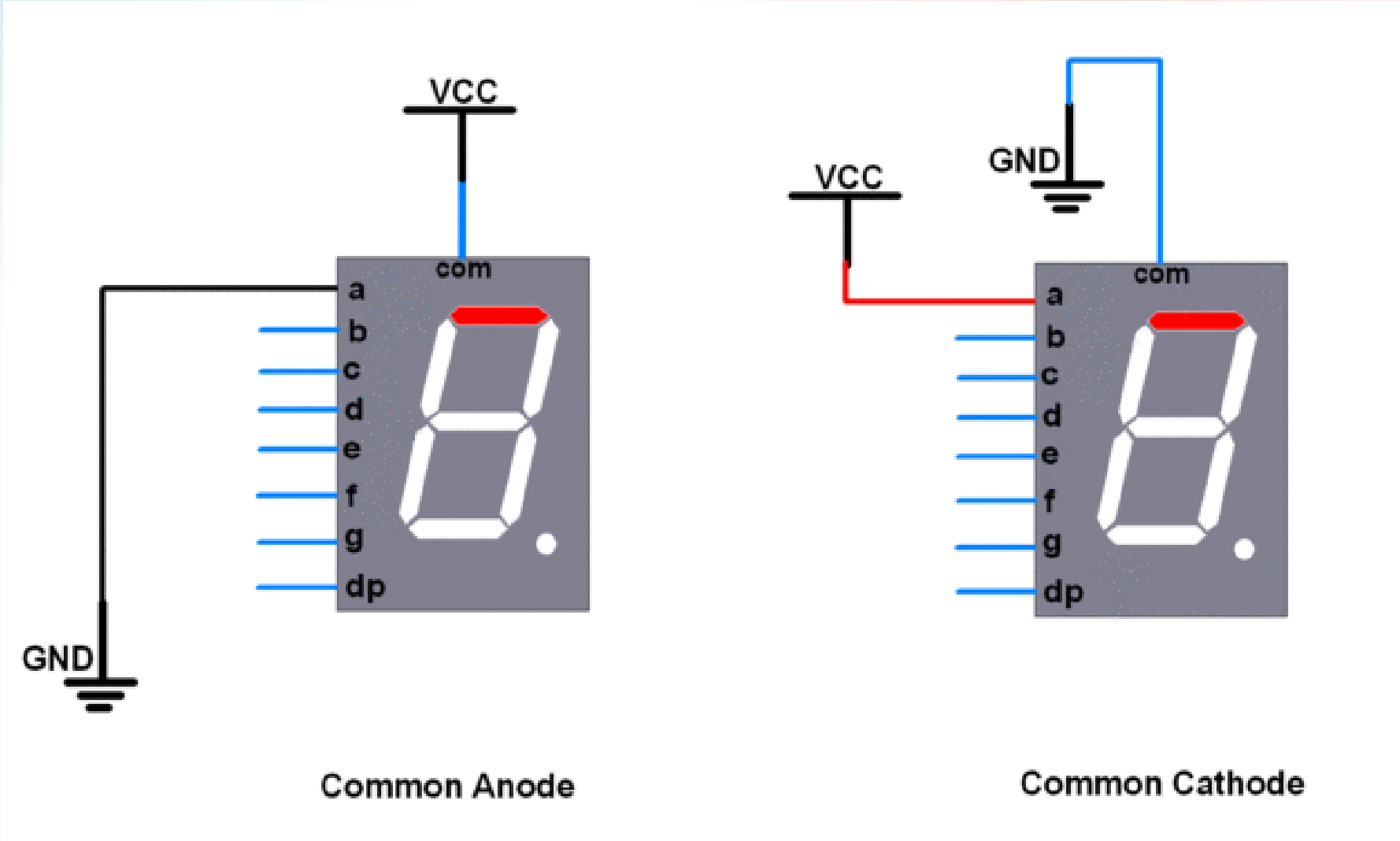
Let's focus on **Common Cathode**.

- Every segment is labeled **a** through **g**.
- We usually represent them as a 7-bit bus: [6:0] segments.

```
seg[0] = a
seg[1] = b
seg[2] = c
seg[3] = d
seg[4] = e
seg[5] = f
seg[6] = g
```

3. The Seven Nation Army

```
seg[0] = a
seg[1] = b
seg[2] = c
seg[3] = d
seg[4] = e
seg[5] = f
seg[6] = g
```



3. The Seven Nation Army

A **seven segment display encoder**

- takes in a 4-bit binary number.
- Gets out a 7-biut control signal [one bit per bar]

Input

4 bits = 2^4 = 16 possibilities

ie, numbers 0-9

```
input [3:0] hex_digit
```

Output

Each bit on the bus is a power line to one of 7 LED's.

```
output reg [6:0] segments
```

3. A syntax note

A physical bundle of 4 wires

```
input [3:0] hex_digit
```

System input (duh)

In the container ‘hex_digit’

```
hex_digit[0]
hex_digit[1]
hex_digit[2]
hex_digit[3]
```

3. A syntax note

Hex zero

Seperator

Binary (b)

```
4'h0: seg = 7'b0111111;
```

7 bits

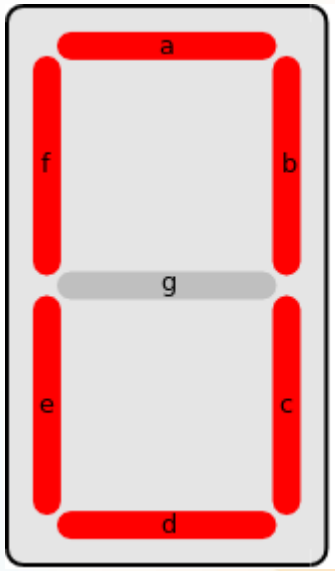
One per segment

Size is 4 bits

seperator

0111111

Shape of zero



a	1
b	1
c	1
d	1
e	1
f	1
g	0

3. The Seven Nation Army

```
module hex_to_7seg (
    input [3:0] hex_digit,   // 4 wires coming in (0-15)
    output reg [6:0] seg     // 7 wires going out (a, b, c, d, e, f, g)
);

    // This block "watches" the input wires.
    // If any wire changes, it re-calculates the output immediately.
    always @(*) begin
        case (hex_digit)
            // Pattern: {g, f, e, d, c, b, a}

            // For 0: All outer bars ON, middle bar (g) OFF
            4'h0: seg = 7'b0111111;

            // For 1: Only bars 'b' and 'c' are ON
            4'h1: seg = 7'b0000110;

            // --- CLASS TASK: FILL IN 2 THROUGH 9 ---

            default: seg = 7'b0000000; // Turn everything off if unknown
        endcase
    end

endmodule
```

3. The Seven Nation Army (Answer)

```
always @(*) begin
    case (hex_digit)
        // Patterns: {g, f, e, d, c, b, a}
        4'h0: seg = 7'b0111111; // 0: All but middle (g)
        4'h1: seg = 7'b0000110; // 1: Only right side (b, c)
        4'h2: seg = 7'b1011011; // 2: a, b, g, e, d
        4'h3: seg = 7'b1001111; // 3: a, b, g, c, d
        4'h4: seg = 7'b1100110; // 4: f, g, b, c
        4'h5: seg = 7'b1101101; // 5: a, f, g, c, d
        4'h6: seg = 7'b1111101; // 6: a, f, e, d, c, g
        4'h7: seg = 7'b0000111; // 7: a, b, c
        4'h8: seg = 7'b1111111; // 8: All segments ON
        4'h9: seg = 7'b1101111; // 9: a, b, c, d, f, g

        // Bonus: Hexadecimal
        4'hA: seg = 7'b1110111; // A
        4'hB: seg = 7'b1111100; // b (usually lowercase to distinguish from 8)
        4'hC: seg = 7'b0111001; // C
        4'hD: seg = 7'b1011110; // d (usually lowercase to distinguish from 0)
        4'hE: seg = 7'b1111001; // E
        4'hF: seg = 7'b1110001; // F

        default: seg = 7'b0000000; // Safety: All OFF
    endcase
end
```

In Hardware: The `*` tells the simulator: "Whenever *any* signal on the `hex_digit` wires changes (even for a picosecond), immediately re-calculate the `seg` wires."

3. The Seven Nation Army (Answer)

Now try with the testbench on Github

4. The 50Mhz Problem

4. What's the Frequency, Kenneth?

- MOST FPGA's you'll find use a 50MHz Clock Signal
- MOST Humans like to count in seconds

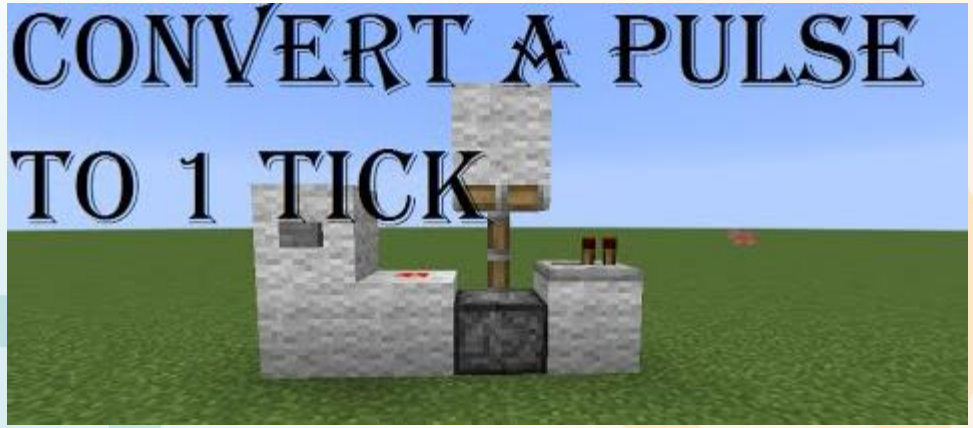
- We want to change a tick to one every second

- We need to build a **counter**

PiezoElectric Quartz Oscillating Crystal

energy

(shakes) At 50Mhz



4. What's the Frequency, Kenneth?

- You need to make a bouncer that will only let every 50 millionth person in.

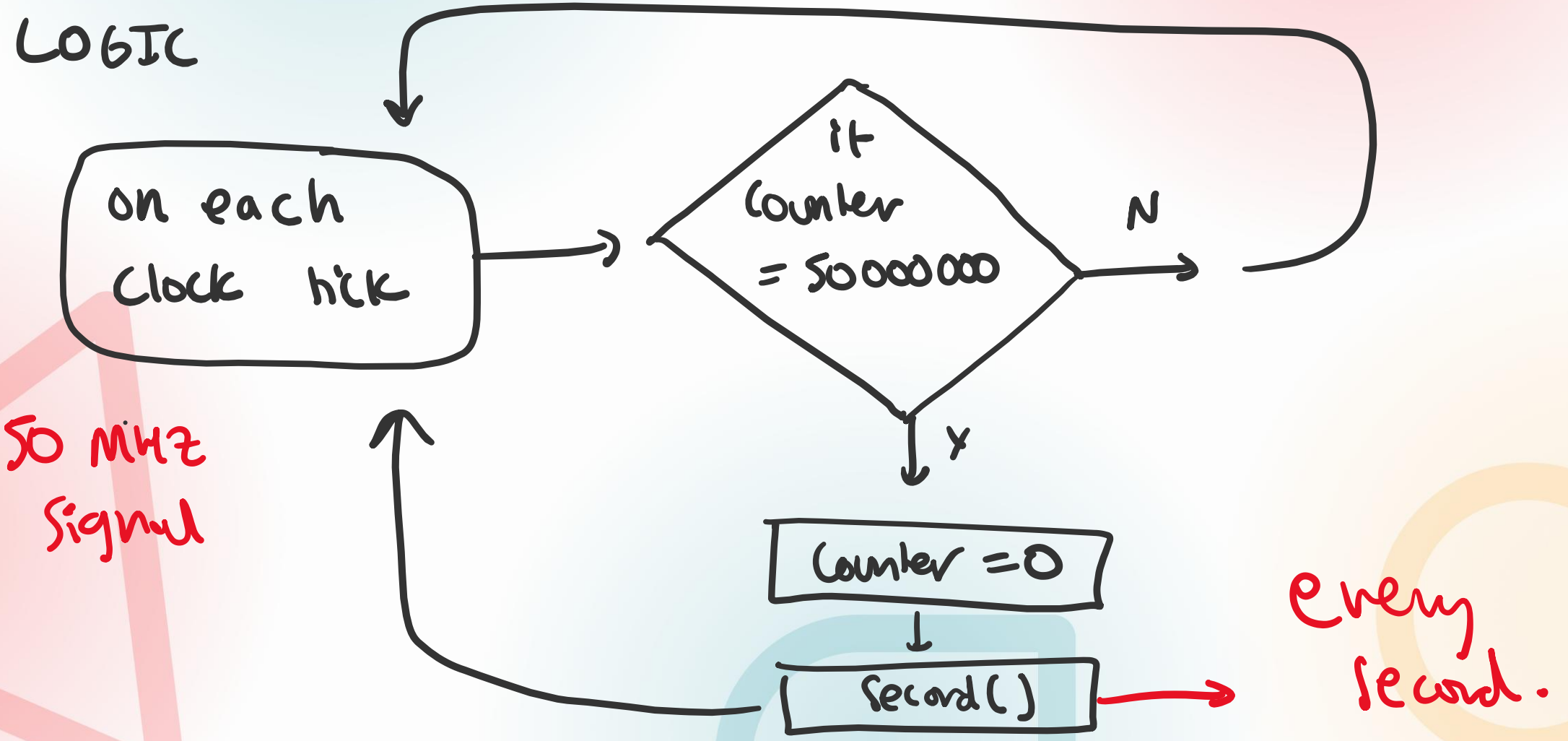
1. **Let's make a register big enough**

$2^{25} = 33\ 554\ 432$ (Small)

$2^{26} = 67\ 108\ 864$ (✓)

4. What's the Frequency, Kenneth?

2. LOGIC



4. What’s the Frequency, Kenneth?

- ‘Wires’ syntax to flip-flops syntax ‘assign’

```
=
```

The Blocking Assignment (=)

"Blocking" (because it evaluates line-by-line, like software).

A Wire.

Immediate. As soon as the input changes, the output changes.

Where to use it: Inside always @(*) blocks (Combinational Logic).

Analogy: A light switch. You flip it, the light goes on. No waiting.

```
<  =
```

The Non-Blocking Assignment (<=)

"Non-Blocking."

A Flip-Flop (Memory).
Delayed. It looks at the value *now*, but doesn't update the output until the **next** clock tick.

Where to use it: Inside always @(posedge clk) blocks (Sequential Logic).

Analogy: A digital camera. You press the shutter (the clock edge), and it captures whatever was in front of it at that exact moment.

4. What’s the Frequency, Kenneth?

- ‘Wires’ vs ‘Assign’, test your understanding

```
// BLOCKING (=)
always @(*) begin
    A = 1;
    B = A; // B becomes 1 immediately
end

// NON-BLOCKING (< =)
always @(posedge clk) begin
    A < = 1;
    B < = A; // B becomes the OLD value of A!
end
```

4. What's the Frequency, Kenneth?

- Try to fill in the gaps

```
reg [25:0] count; // Our 26-bit counter
reg [3:0]  value; // The 4-bit number for our 7-seg

always @(posedge clk) begin
    if (count == 50_000_000) begin
        [redacted]                  // Reset counter
        [redacted]                  // Increment the display digit
    end else begin
        [redacted]                  // Keep waiting
    end
end
```

4. What’s the Frequency, Kenneth?

- Now check your solution

```
reg [25:0] count; // Our 26-bit counter
reg [3:0]  value; // The 4-bit number for our 7-seg

always @(posedge clk) begin
    if (count == 50_000_000) begin
        count < = 0;           // Reset counter
        value < = value + 1;   // Increment the display digit
    end else begin
        count < = count + 1;   // Keep waiting
    end
end
```

4. What’s the Frequency, Kenneth?

Now try with the testbench on Github

5. Connecting FPGA's to other FPGA's

5. OK COMPUTER

- We'll combine our seven segment and counter to make a stopwatch

```
module top (
    input clk,              // The 50MHz physical pin
    output [6:0] seg_out    // The 7 physical LED pins
);

    // This is the physical "copper trace" connecting the two modules
    wire [3:0] internal_number;

    // Call the Counter
    counter_example my_timer (
        .clk(clk),
        .value(internal_number) // Output of timer goes TO the wire
    );

    // Call the Decoder
    hex_to_7seg my_display (
        .hex_digit(internal_number), // Input of decoder comes FROM the wire
        .seg(seg_out)
    );

endmodule
```

```
> iverilog -o system_test top.v counter_example.v sevenseg_example.v
tb_top.v
> vvp system_test
```

5. OK COMPUTER

Now try with the testbench on Github

6. Homework

6. One for the road

- Make the stopwatch count down from 100
- Add a reset button signal